Programación por bloques

Nombre: _____ Fecha: _____

Qué vas a aprender

- Explicar qué es un algoritmo y representarlo con un diagrama de flujo
- Conocer el entorno de un lenguaje de programación por bloques
- Usar secuencias, bucles, condicionales, variables y eventos
- Diseñar, probar y depurar programas sencillos

Para pensar

Los programadores profesionales escriben código con palabras y símbolos. ¿Por qué entonces se aprende a programar con bloques de colores que encajan como piezas?

Explicación

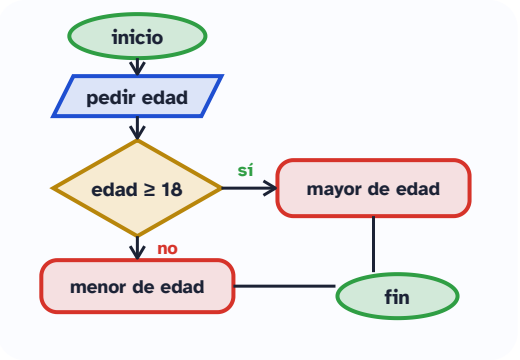
Algoritmos y programas

Un algoritmo es una **secuencia ordenada y finita de instrucciones** que resuelve un problema o realiza una tarea. Las recetas de cocina, las instrucciones de montaje de un mueble o los pasos para resolver una ecuación son algoritmos. Un buen algoritmo debe ser **preciso** (cada paso claro, sin ambigüedades), **ordenado** y **finito** (tiene que terminar). Un programa es un algoritmo escrito en un **lenguaje de programación** para que lo ejecute un ordenador. El ordenador no piensa ni adivina: hace exactamente lo que se le dice, en el orden en que se le dice. Si las instrucciones están mal o en mal orden, el resultado será incorrecto. Por eso antes de programar conviene pensar bien el algoritmo, escribirlo en lenguaje natural o dibujarlo.

undefined

Diagramas de flujo y pseudocódigo

Para planificar un algoritmo se usan dos herramientas. El diagrama de flujo es un dibujo con **símbolos normalizados** unidos por flechas que indican el orden: el **óvalo** marca el inicio y el fin; el **rectángulo**, una acción o proceso; el **rombo**, una decisión con dos salidas (sí o no); y el **paralelogramo**, una entrada o salida de datos (pedir un número, mostrar un mensaje). El pseudocódigo es una forma de escribir el algoritmo con palabras sencillas, parecidas a las de un lenguaje de programación, pero sin preocuparse por las reglas exactas: «pedir edad; si edad ≥ 18 entonces decir "mayor de edad"; si no, decir "menor de edad"». Ambas herramientas ayudan a detectar errores antes de programar y a explicar a otras personas cómo funciona un programa.



Los lenguajes por bloques

En los lenguajes de programación por bloques las instrucciones son piezas de colores que se arrastran y se encajan como un puzle. El más conocido es **Scratch**, creado en el MIT, gratuito y que se usa en el navegador; también hay otros como **MakeCode** (para placas micro:bit), **mBlock** (para robots), **App Inventor** (para crear aplicaciones de móvil) o **Blockly**. En Scratch, la pantalla tiene tres zonas: a la izquierda, la **paleta de bloques**, ordenada por categorías de colores (movimiento, apariencia, sonido, eventos, control, sensores, operadores y variables); en el centro, el **área de programación**, donde se montan los programas; y a la derecha, el **escenario**, donde se mueven los **objetos** o personajes. Cada objeto tiene sus propios programas, disfraces y sonidos, y el escenario puede tener varios fondos.

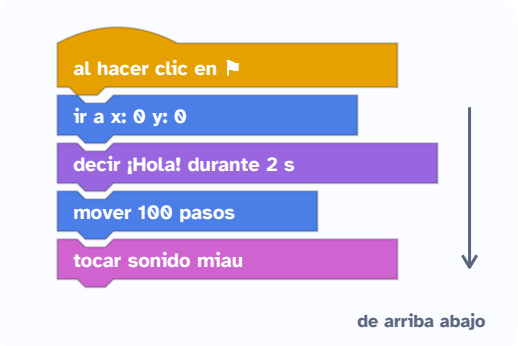
Categoría	Color	Para qué sirve
Movimiento	Azul	Mover, girar, ir a x, y

Categoría	Color	Para qué sirve
Apariencia	Morado	Decir, cambiar disfraz, tamaño
Sonido	Rosa	Tocar sonidos
Eventos	Amarillo	Empezar programas
Control	Naranja	Esperar, repetir, si...
Sensores	Azul claro	¿Tocando?, preguntar
Operadores	Verde	Sumas, comparaciones, azar
Variables	Naranja oscuro	Guardar datos

undefined

Eventos y secuencias

Todo programa en Scratch empieza con un evento: un bloque amarillo con forma de «sombrero» que indica **cuándo** se ejecuta el programa. Los más usados son «al hacer clic en la bandera verde», «al presionar la tecla espacio», «al hacer clic en este objeto» o «al recibir mensaje». Debajo del evento se encajan las instrucciones en secuencia: se ejecutan **una detrás de otra, de arriba abajo**, en el orden en que están colocadas. Por ejemplo: al hacer clic en la bandera, ir a x: 0, y: 0; decir «¡Hola!» durante 2 segundos; mover 100 pasos; tocar sonido «miau». El orden importa: si pones «mover» antes de «ir a x: 0, y: 0», el gato se moverá pero volverá al centro enseguida y no verás el movimiento. Además, un mismo objeto puede tener varios programas que se ejecutan a la vez, cada uno con su evento.



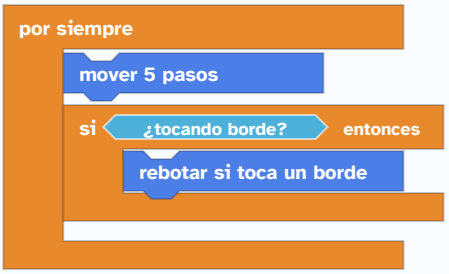
Bucles: repetir instrucciones

Muchas veces hay que repetir lo mismo varias veces. En lugar de copiar los bloques, se usa un bucle, un bloque de **control** (naranja) con forma de C que abraza las instrucciones que se repiten. Hay tres tipos. **«Repetir 10»** repite un número fijo de veces; por ejemplo, para dibujar un cuadrado con el lápiz: repetir 4 veces { mover 100 pasos; girar 90 grados }. **«Por siempre»** repite sin fin, mientras el programa esté en marcha; es el que se usa en los videojuegos para comprobar constantemente si se pulsa una tecla o si hay un choque. **«Repetir hasta que…»** repite mientras no se cumpla una condición; por ejemplo, avanzar hasta tocar el borde. Los bucles hacen los programas más **cortos**, más fáciles de leer y de cambiar: si quieres un hexágono, solo cambias 4 por 6 y 90 por 60.



Condicionales: tomar decisiones

Un programa necesita a veces decidir qué hacer según lo que ocurra. Para eso están los condicionales, también bloques de control con forma de C. El bloque **«si… entonces»** ejecuta las instrucciones de dentro solo si se cumple una **condición**; si no se cumple, se las salta. El bloque **«si… entonces… si no»** tiene dos huecos: uno para cuando la condición es verdadera y otro para cuando es falsa. Las condiciones son bloques con forma hexagonal que solo pueden valer **verdadero o falso**: preguntas de sensores, como «¿tocando borde?», «¿tecla espacio presionada?», «¿tocando color rojo?»; o comparaciones de operadores, como «puntos > 10» o «respuesta = 7». Se pueden combinar con «y», «o» y «no». Un condicional casi siempre va dentro de un bucle «por siempre», para que la pregunta se haga continuamente.



Variables y operadores

Una variable es como una **caja con nombre** en la que el programa guarda un dato que puede cambiar: los puntos de un juego, las vidas, el tiempo, la velocidad o la respuesta del usuario. En Scratch se crean en la categoría «Variables» y se usan con tres bloques: **«dar a puntos el valor 0»** (al empezar el juego), **«sumar a puntos 1»** (cada vez que se consigue algo) y el bloque redondo **«puntos»**, que se puede colocar dentro de otros bloques para usar su valor. Las variables pueden mostrarse en el escenario como un marcador. Los operadores (verdes) permiten hacer cálculos (+, −, ×, ÷), comparar (>, <, =), elegir un número al azar o unir textos. Por ejemplo, «si puntos = 10 entonces decir "¡Has ganado!"». Con un bloque «preguntar» y la variable «respuesta» se pueden hacer programas que dialogan con el usuario, como un juego de preguntas.

undefined

Probar y depurar

Casi ningún programa funciona bien a la primera, ni siquiera los de los programadores profesionales. Los errores se llaman bugs («bichos» en inglés) y el proceso de encontrarlos y corregirlos se llama depurar. Algunas estrategias útiles: **probar a menudo**, después de añadir cada pocos bloques, en vez de escribir todo el programa y probarlo al final; **leer el programa paso a paso** como si fueras el ordenador; añadir bloques «decir» o mostrar variables para ver qué valor tienen; probar casos raros (¿qué pasa si el jugador pulsa dos teclas a la vez?); y **pedir a un compañero** que lo pruebe, porque encontrará fallos que tú no ves. Los errores más frecuentes en Scratch son olvidar el evento, colocar los bloques en mal orden, no reiniciar las variables al empezar o poner un condicional fuera del bucle «por siempre».

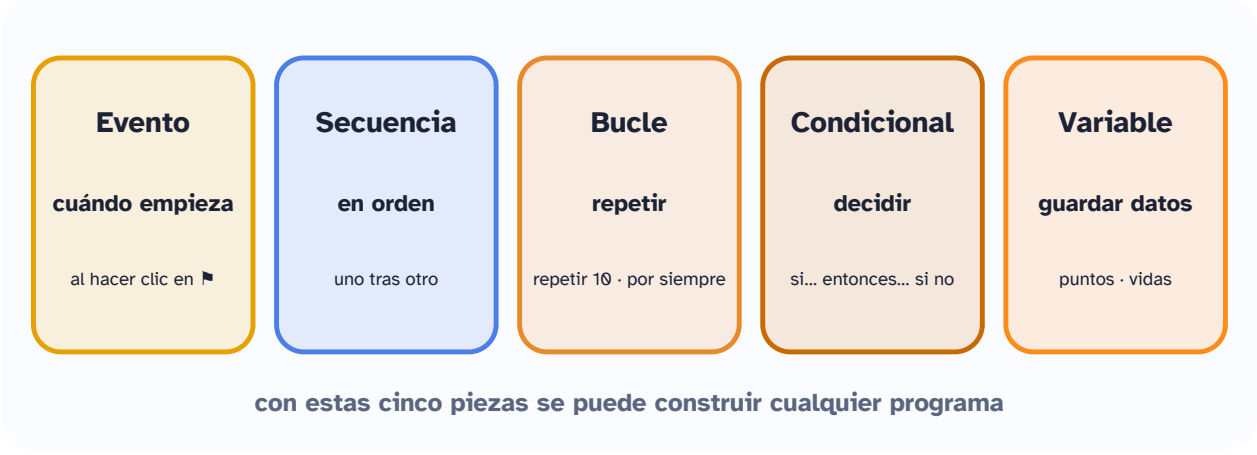
undefined

Del bloque al código y la programación en la vida real

Lo que aprendes con bloques sirve después en cualquier lenguaje escrito. Por ejemplo, el bucle «repetir 4 { mover 100; girar 90 }» se escribe en Python así: `for i in range(4): forward(100); right(90)`. Y el condicional «si puntos > 10 entonces decir "¡Bien!"» se escribe `if puntos > 10: print("¡Bien!")`. La programación está en todas partes: en los móviles, los electrodomésticos, los coches, los semáforos, los videojuegos, la medicina y la ciencia. Además, programar desarrolla el pensamiento computacional: la capacidad de **descomponer** un problema grande en partes pequeñas, **reconocer patrones** que se repiten, **abstraer** lo importante y diseñar **algoritmos** para resolverlo. Esa forma de pensar es útil en cualquier profesión, no solo en la informática.

undefined

Las estructuras básicas de la programación



Ejercicios

1. **Define algoritmo y pon un ejemplo de la vida diaria.**

2. **¿Qué símbolos usa un diagrama de flujo para inicio/fin, proceso y decisión?**

3. **Escribe en pseudocódigo: si llueve, coger paraguas; si no, coger gorra.**

4. **Nombra las tres zonas de la pantalla de Scratch.**

5. **¿Qué hace un bloque de evento y de qué color es?**

6. **¿Cuántas veces hay que repetir y cuántos grados girar para un hexágono?**

7. **Diferencia «repetir 10» y «por siempre».**

8. **¿Qué valores puede tomar una condición?**

9. **¿Para qué sirve una variable? Pon un ejemplo en un juego.**

10. **¿Qué es depurar y nombra dos estrategias?**

Practica

1. **Un algoritmo debe ser...**
   - **A)** infinito
   - **B)** preciso, ordenado y finito
   - **C)** secreto
2. **En un diagrama de flujo la decisión es un...**
   - **A)** óvalo
   - **B)** rombo
   - **C)** rectángulo
3. **Scratch se creó en el...**
   - **A)** MIT
   - **B)** CERN
   - **C)** NASA
4. **Los bloques de eventos son...**
   - **A)** azules
   - **B)** amarillos
   - **C)** verdes
5. **Las instrucciones en secuencia se ejecutan...**
   - **A)** a la vez
   - **B)** de arriba abajo
   - **C)** al azar
6. **Para dibujar un cuadrado se repite...**
   - **A)** 3 veces 120°
   - **B)** 4 veces 90°
   - **C)** 6 veces 60°
7. **Los videojuegos usan mucho el bucle...**
   - **A)** repetir 2
   - **B)** por siempre
   - **C)** ninguno
8. **Una condición vale...**
   - **A)** un número

B) verdadero o falso

C) un texto

9. **Los puntos de un juego se guardan en...**

A) una variable

B) un disfraz

C) un fondo

10. **Un bug es...**

A) un error

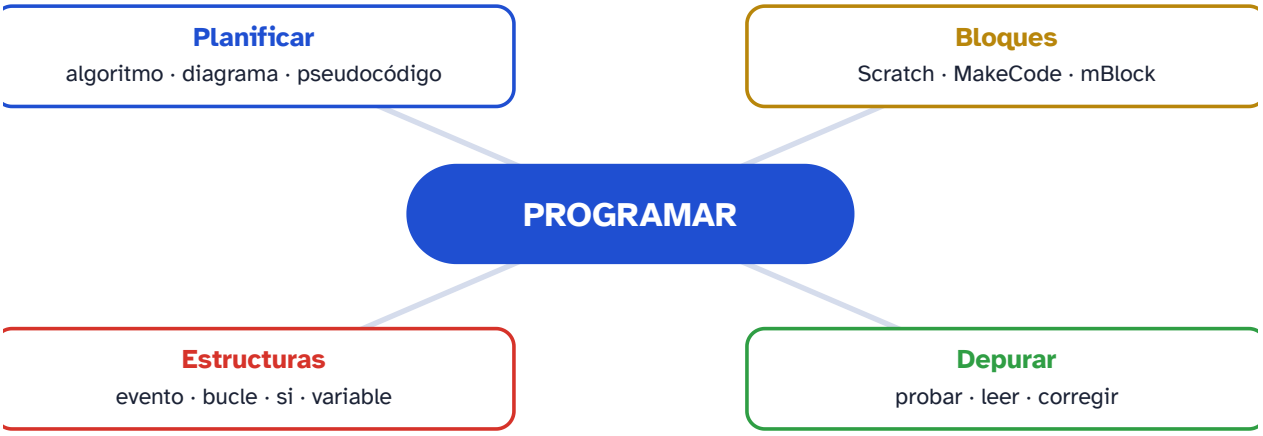
B) un bloque

C) un objeto

Explícalo con tus palabras

Incluye: Qué es un algoritmo y cómo se dibuja un diagrama de flujo; Para qué sirven los bucles y los condicionales; Qué es una variable y cómo se usa en un juego.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Algoritmo

Instrucciones ordenadas y finitas para resolver un problema.

Diagrama de flujo

Dibujo del algoritmo con símbolos y flechas.

Evento

Bloque que indica cuándo empieza un programa.

Secuencia

Instrucciones una tras otra, de arriba abajo.

Bucle

Repite instrucciones: repetir, por siempre, hasta que.

Condicional

Si… entonces… si no: decide.

Variable

Caja con nombre que guarda un dato que cambia.

Depurar

Encontrar y corregir errores (bugs).

Soluciones

1. **B)** Preciso, ordenado y finito.
2. **B)** Rombo.
3. **A)** MIT.
4. **B)** Amarillos.
5. **B)** Una tras otra.
6. **B)** $4 \cdot 90° = 360°$.
7. **B)** Comprueban todo el rato.
8. **B)** Booleana.
9. **A)** Variable.
10. **A)** Un fallo del programa.

Soluciones de los ejercicios

1. Secuencia ordenada y finita de instrucciones para resolver un problema: una receta.
2. Óvalo, rectángulo y rombo.
3. si llueve entonces coger paraguas; si no coger gorra; fin si.
4. Paleta de bloques, área de programación y escenario.
5. Indica cuándo empieza el programa; es amarillo.
6. 6 veces, 60 grados.
7. El primero repite un número fijo de veces; el segundo repite sin fin.
8. Verdadero o falso.
9. Para guardar un dato que cambia: los puntos o las vidas.
10. Encontrar y corregir errores: probar a menudo y leer el programa paso a paso.